MongoDB Relationships

Embedding vs Referencing — explained simply

The Basic Question

MongoDB stores data as separate **documents** inside **collections** (for example, separate Student documents inside a “students” collection). A “relationship” is simply: how do two related documents know about each other? There are only two ways to do this.

Way 1 — Embedding (put the data INSIDE the document)

```
{
name: "Rahul",
address: { city: "Pune", street: "MG Road" }
}
```

The address lives right inside the student document. One single read gets you everything — no second lookup needed.

Use embedding when the extra data:

- is small
- always belongs to just **one** parent document
- is always read together with the parent (never needed on its own)

Way 2 — Referencing (store just an ID, not the data)

```
// students collection
{ name: "Rahul", course: "65f0a1..." } // just an id, like a pointer

// courses collection
{ _id: "65f0a1...", title: "CS", credits: 4 }
```

The student document only stores the Course's **_id**. The actual course data lives on its own, in a separate collection.

Use referencing when the extra data:

- is shared by **many** parents (100 students, 1 “CS” course — no point copying the same course info 100 times)
- has its own independent life (a course still exists even with zero students enrolled)
- can grow very large over time

How You Get The Real Data Back — populate()

A reference alone only gives you an id, not the actual course info. **.populate()** is the command that goes and fetches the real data for you.

```
this.studentModel.findById(id).populate('course').exec();
```

Before populate	`course: "65f0a1..."`
After populate	`course: { title: "CS", credits: 4 }`

Think of **populate()** as: “go look up what this id actually points to.”

The Three Relationship Shapes

Shape	Example	Usual approach
One-to-One	one student, one address	usually embed
One-to-Many	one course, many students in it	reference (course stores nothing, or an array of student ids)
Many-to-Many	many students, many courses each	reference — each side stores an array of the other side's ids

1. One-to-One — in detail

One student has exactly one address, and that address belongs to only that student. Since it's small and always read together with the student, just **embed** it.

```
@Schema()
class Address {
@Prop() street!: string;
@Prop() city!: string;
}

@Schema({ timestamps: true })
class Student {
@Prop({ required: true }) name!: string;
@Prop({ type: Address }) address!: Address; // embedded, not a separate collection
}

// resulting document — one read gets everything:
{
name: "Rahul",
address: { street: "MG Road", city: "Pune" }
}
```

If the related data instead needs its OWN identity (e.g. a Profile document that must be queried on its own sometimes), reference it with a single ObjectId instead of an array:

```
@Prop({ type: Types.ObjectId, ref: 'Profile' })
profile!: Types.ObjectId;
```

2. One-to-Many — in detail

One-to-Many has BOTH an embedded version and a referenced version — which one to use depends on whether the “many” side is small/bounded or large/independent.

2a. One-to-Many — EMBEDDED

One Student has many Addresses (home, hostel, office) — small, bounded, never queried on their own. Embed them as an array directly inside the parent document:

```
@Schema()
class Address {
@Prop() label!: string; // e.g. "home", "hostel"
@Prop() city!: string;
}

@Schema({ timestamps: true })
export class Student {
@Prop({ required: true }) name!: string;

@Prop({ type: [Address] })
addresses!: Address[]; // embedded array — lives INSIDE the student doc
}

// resulting document — one read gets every address:
{
name: "Rahul",
addresses: [
```

```
{ label: "home", city: "Pune" },
{ label: "hostel", city: "Mumbai" }
]
}
```

No second query, no populate() needed — the whole array comes back with a plain **.findById()**. Use this when the list per parent stays small (a handful of addresses, not thousands of orders).

2b. One-to-Many — REFERENCED

One Course has many Students enrolled in it — potentially hundreds, and each Student exists independently of any one course. Store a reference instead of copying data. Simplest direction: the reference lives on the “many” side (each Student points at one Course):

```
@Schema({ timestamps: true })
export class Student {
@Prop({ required: true }) name!: string;

@Prop({ type: Types.ObjectId, ref: 'Course' })
course!: Types.ObjectId; // many students -> point at the same one course
}
```

Reading it back with the real course data filled in:

```
await this.studentModel.findById(id).populate('course').exec();
// -> { name: "Rahul", course: { title: "CS", credits: 4 } }
```

Sometimes you need it the OTHER way — given a Course, list all its Students. Store an array of references on the “one” side instead:

```
@Schema({ timestamps: true })
export class Course {
@Prop({ required: true }) title!: string;

@Prop({ type: [Types.ObjectId], ref: 'Student' })
students!: Types.ObjectId[]; // one course -> many student ids
}

await this.courseModel.findById(id).populate('students').exec();
// -> { title: "CS", students: [ {Student doc}, {Student doc}, ... ] }
```

3. Many-to-Many — in detail

Many students can each take many courses. Simplest approach: an array of references on **both** sides.

```
// student.schema.ts
@Prop({ type: [Types.ObjectId], ref: 'Course' })
courses!: Types.ObjectId[];

// course.schema.ts
@Prop({ type: [Types.ObjectId], ref: 'Student' })
students!: Types.ObjectId[];
```

Trade-off: YOU must keep both arrays in sync yourself (add the course's id to the student's array AND the student's id to the course's array) — Mongo won't do it automatically.

If the relationship itself needs extra data (e.g. an enrollment date or a grade), give it its own collection instead — same idea as a SQL join table:

```
@Schema({ timestamps: true })
export class Enrollment {
@Prop({ type: Types.ObjectId, ref: 'Student', required: true })
student!: Types.ObjectId;

@Prop({ type: Types.ObjectId, ref: 'Course', required: true })
course!: Types.ObjectId;

@Prop() grade!: string;
}
```

The One Thing To Remember

MongoDB does **NOT** check that a reference is valid. You can store a course id that doesn't exist, and nothing stops you. Unlike SQL's **FOREIGN KEY**, keeping references correct is 100% your responsibility in application code.

> **Rule of thumb:** Small + always-together + belongs to one parent → **embed** it.
> Shared + independent + grows big → **reference** it, then **populate()** when you actually need the real data.